

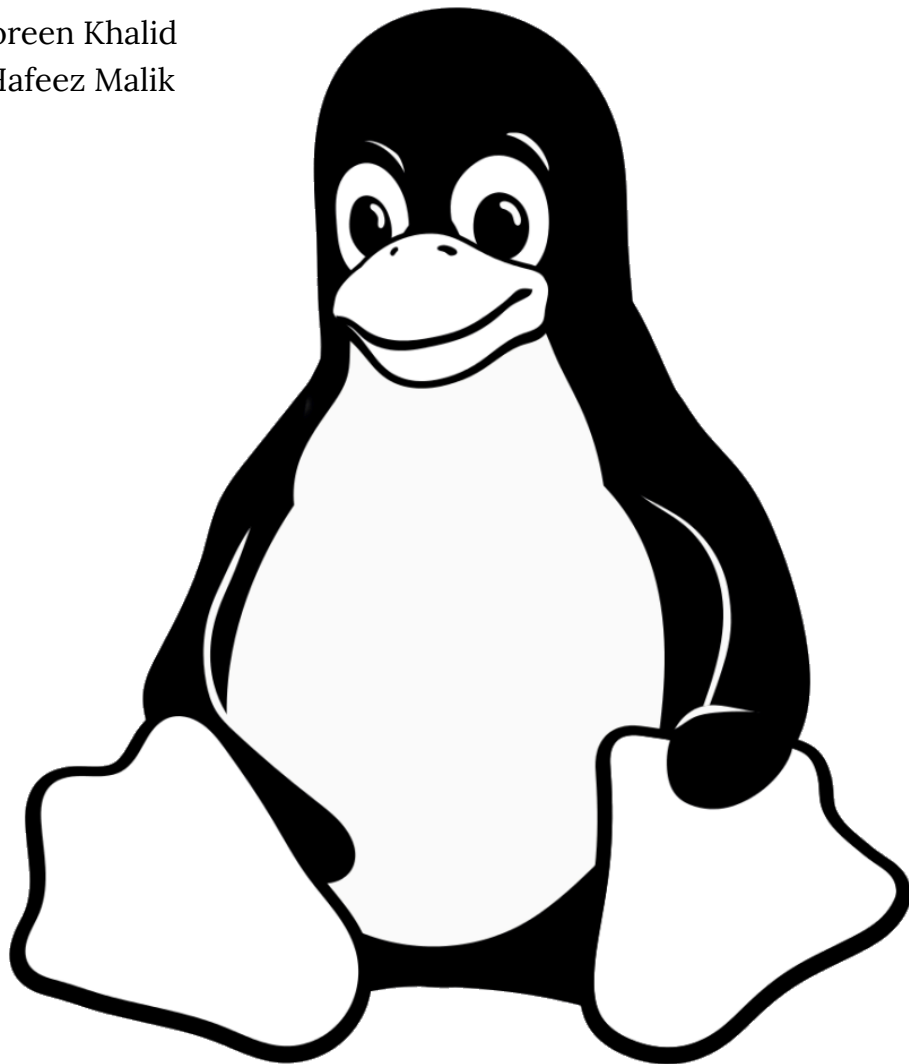


Saad Mughal
F2023-009
Total Marks: 10
Due 14 November 2025

Operating Systems LAB

Assignment 01

Course Lecturer: Noreen Khalid
Lab Instructor: Ali Hafeez Malik
Section: C



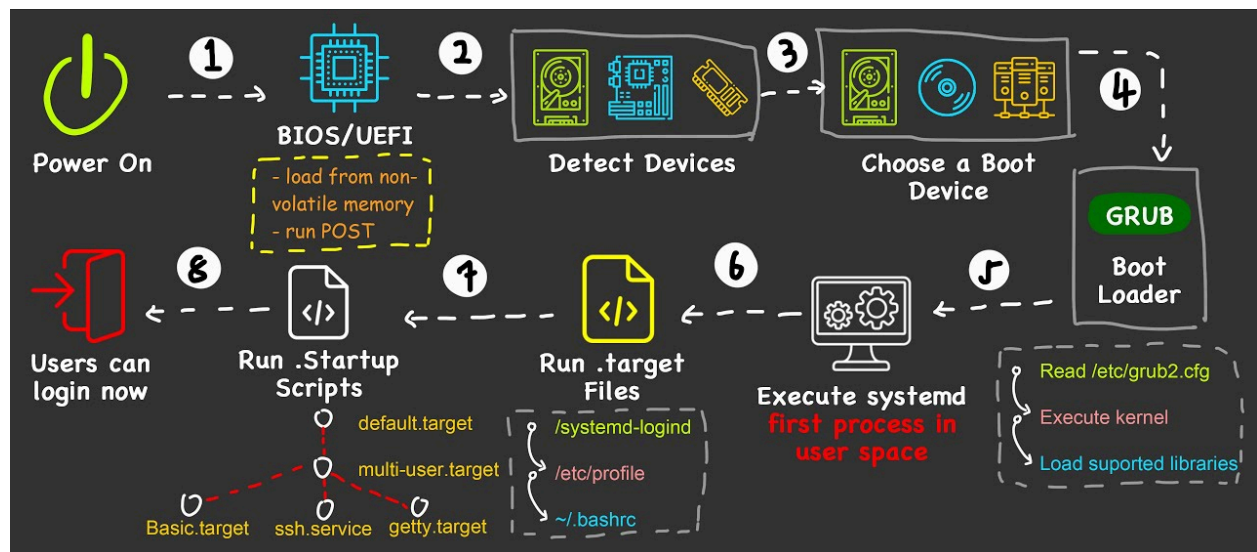
Operating Systems LAB (LAB-217) - Assignment 01

Linux Boot Process & Linux Directory System

SECTION I: Linux Boot Process

Overview

The Linux boot process transforms a powered-off computer into a fully operational system through seven systematic stages.



1. Power-On and Hardware Initialization

The Power Supply Unit (PSU) distributes power to hardware components. The CPU receives power, resets to default state, and begins fetching its first instruction from firmware (BIOS/UEFI) stored in ROM.

2. Firmware: BIOS vs UEFI

BIOS (Basic Input/Output System)

- Legacy 16-bit technology
- Uses MBR partitioning (max 2.2 TB disk)
- Text-based interface

- Loads first 512 bytes containing bootloader

UEFI (Unified Extensible Firmware Interface)

- Modern 32/64-bit standard
- Supports GPT partitioning (no size limit)
- Graphical interface with mouse support
- Faster boot times with Secure Boot capability
- Directly executes bootloader from EFI System Partition

3. POST (Power-On Self Test)

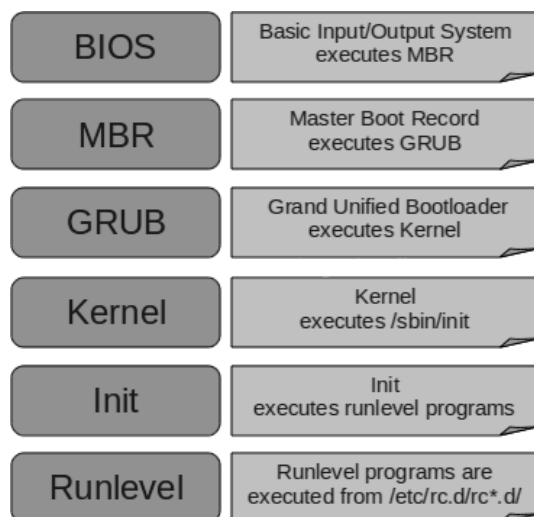
Diagnostic testing sequence verifying hardware functionality:

- **Components tested:** CPU, RAM, video card, keyboard/mouse, storage devices, peripherals
- **Error communication:** Beep codes, on-screen messages, diagnostic LEDs
- System halts if critical problems detected; otherwise proceeds to bootloader

4. Bootloader: GRUB

GRUB Boot Stages

1. **Stage 1 (boot.img):** Minimal code in MBR/EFI partition
2. **Stage 1.5 (core.img):** Contains filesystem drivers (BIOS only)
3. **Stage 2:** Full GRUB environment in `/boot/grub/`
 - Interactive menu for OS selection
 - Reads `/boot/grub/grub.cfg`
 - Loads kernel and initramfs into memory



Key Configuration Files

- `/etc/default/grub` - User settings
- `/boot/grub/grub.cfg` - Auto-generated (don't edit directly)

Update GRUB:

```
sudo update-grub                                # Debian/Ubuntu
sudo grub2-mkconfig -o /boot/grub2/grub.cfg      # RHEL/Fedora
```

Multi-Boot: GRUB can boot multiple operating systems through chain loading (loading another bootloader like Windows Boot Manager).

5. Linux Kernel Loading

Key Components

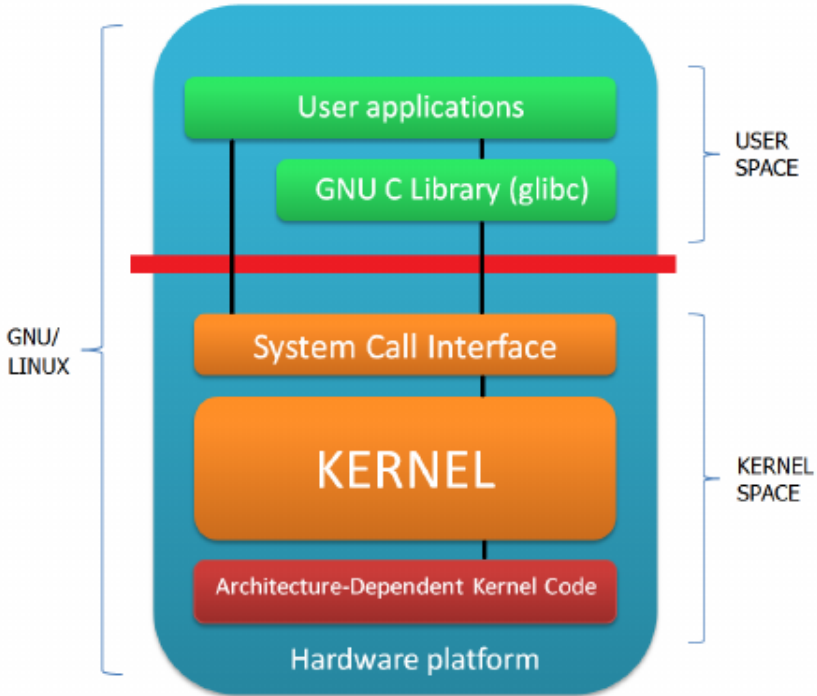
- **Kernel Image:** `/boot/vmlinuz-<version>` (compressed, self-decompressing)
- **Initramfs:** `/boot/initrd.img-<version>` (temporary root filesystem with essential drivers)

Initialization Process

1. Kernel decompresses itself in memory
2. CPU initialization (interrupts, memory management, scheduling)
3. Hardware detection and driver loading
4. Mount initramfs as temporary root
5. Load storage/filesystem drivers from initramfs
6. Mount actual root partition
7. Pivot from initramfs to real root filesystem
8. Start init system (systemd)

Why Initramfs? Solves chicken-and-egg problem: kernel needs drivers to access disk, but drivers are on disk. Initramfs provides drivers in RAM.

6. Kernel Space vs User Space



Aspect	Kernel Space	User Space
Privilege Level	Ring 0 (highest)	Ring 3 (restricted)
Hardware Access	Direct	Via system calls only
Memory Access	Full system	Own process only
Crash Impact	System-wide	Process-level only
Examples	Kernel, drivers	Applications, utilities

System Calls: Bridge between user and kernel space. Examples: `open()`, `read()`, `write()`, `fork()`, `exec()`

7. Init System: Systemd

Systemd (PID 1) is the first user-space process managing system services and boot.

Responsibilities

- Start/stop services in parallel
- Mount filesystems
- Manage user sessions
- System logging (journald)
- Hardware hotplug management

Systemd Targets

Target	Old Runlevel	Description
poweroff.target	0	Shutdown
rescue.target	1	Single-user mode
multi-user.target	3	Multi-user text mode
graphical.target	5	Multi-user with GUI
reboot.target	6	Reboot

Essential Commands

```
# Service Management
sudo systemctl start/stop/restart servicename
sudo systemctl enable/disable servicename
systemctl status servicename

# Target Management
systemctl get-default
sudo systemctl set-default multi-user.target
```

8. Login and User Session

Graphical Login (Desktop)

- Display managers: GDM, LightDM, SDDM
- Start X11/Wayland, authenticate via PAM
- Launch desktop environment (GNOME, KDE, XFCE)

Console Login (Text/Servers)

- Getty spawned on virtual terminals (tty1-tty6)
- PAM authentication
- Start user's default shell (bash)

9. Optional Boot Features

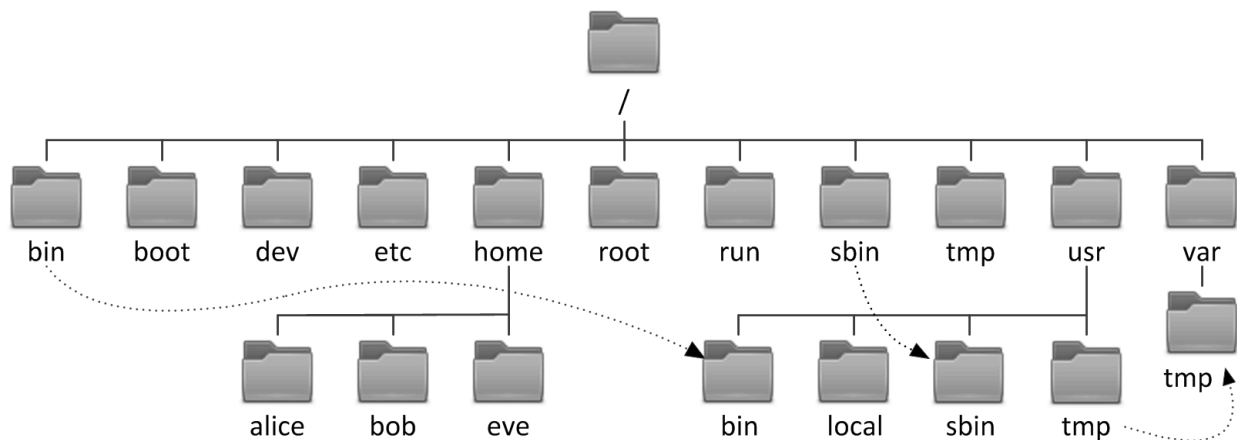
Secure Boot: UEFI feature verifying digital signatures of bootloader/kernel to prevent unauthorized code execution.

Recovery Mode: Available in GRUB menu for troubleshooting with minimal services and read-only root filesystem.

SECTION II: Linux Directory System

Overview

Linux uses a unified tree structure under single root `/` (unlike Windows drive letters). All storage devices mount as subdirectories within this hierarchy following the Filesystem Hierarchy Standard (FHS).



Directory Classification

1. Essential System Directories

`/bin` - Essential Binaries

- Essential commands for boot and single-user mode
- Examples: `ls`, `cp`, `mv`, `cat`, `bash`, `grep`
- Modern systems: often symlinked to `/usr/bin`

`/sbin` - System Binaries

- System administration commands requiring root
- Examples: `fdisk`, `ifconfig`, `iptables`, `reboot`
- Modern systems: often symlinked to `/usr/sbin`

`/boot` - Boot Files

- Critical boot files: **vmlinuz-*** (kernel), **initrd.img-*** (initramfs), **grub/** (bootloader config)
- **Warning:** Do not modify without expertise

/etc - Configuration

- System-wide configuration files
- Key files: **passwd**, **shadow**, **fstab**, **hosts**, **resolv.conf**, **ssh/**

/lib and /lib64 - Libraries

- Shared libraries for **/bin** and **/sbin** binaries
- Similar to Windows .dll files

2. Device and Virtual Directories

/dev - Devices

- Device files (hardware interfaces)
- Examples: **/dev/sda** (disk), **/dev/null** (data sink), **/dev/random** (RNG)
- Virtual filesystem populated by udev

/proc - Process Information

- Virtual filesystem in RAM (not on disk)
- Process and kernel information
- Examples: **/proc/cpuinfo**, **/proc/meminfo**, **/proc/[PID]/**

/sys - System

- Kernel objects and hardware information
- Used by udev for device management

/run - Runtime Data

- Runtime variable data (tmpfs, cleared on reboot)
- PID files, sockets, lock files

3. User and Data Directories

/home - User Home Directories

- Personal directories: **/home/username/**
- User documents, downloads, configurations (**.bashrc**, **.config/**)
- Each user has full control over their directory

/root - Root User Home

- Home directory for root (admin) user
- Separate from **/home** for security
- Available even if **/home** fails to mount

/tmp - Temporary Files

- Temporary files (cleared on reboot or periodically)
- World-writable with sticky bit
- Never store sensitive data here

/var - Variable Data

- Data changing during operation
- Key subdirectories:
 - **/var/log/** - System and application logs
 - **/var/cache/** - Application cache
 - **/var/www/** - Web server root
 - **/var/spool/** - Print/mail/cron queues

4. Removable Media and Mounting

/media - Removable Media

- Automatic mount points for USB drives, external disks, CD/DVD
- Structure: **/media/username/device_label/**

/mnt - Mount

- Temporary manual mount point
- Used by admins for maintenance

5. Optional Software Directories

/opt - Optional Software

- Third-party applications (Google Chrome, proprietary software)
- Self-contained packages: **/opt/package_name/**

/srv - Service Data

- Data for system-provided services (FTP, web servers)

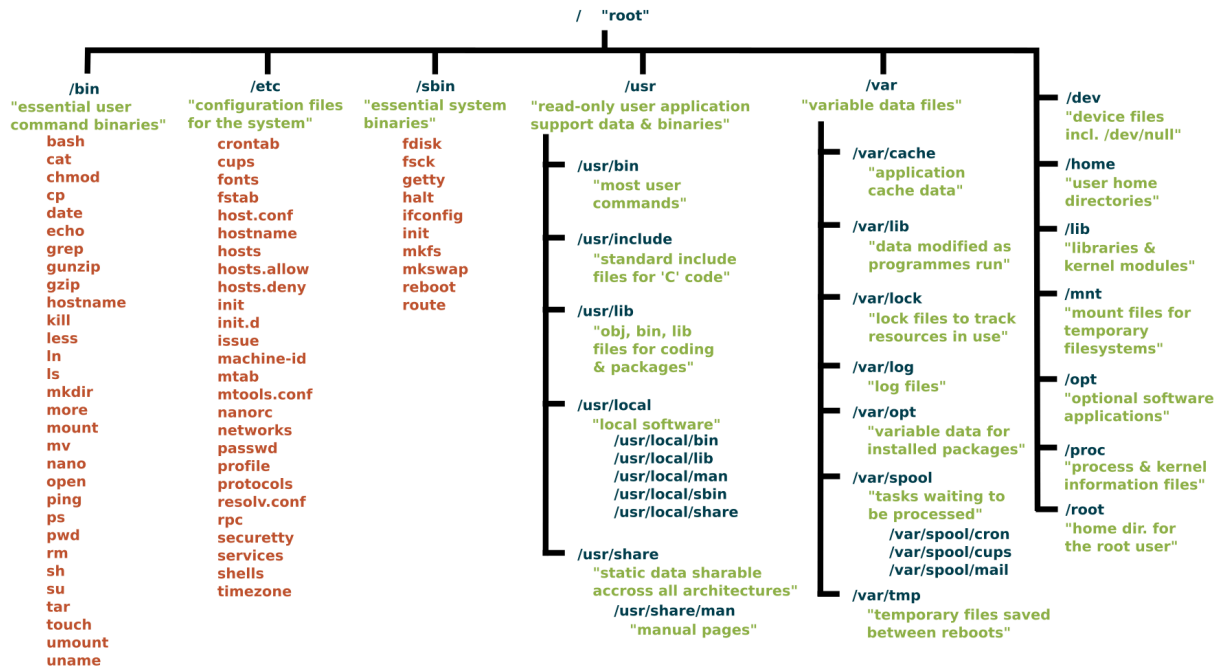
6. User Programs Directory

/usr - User Programs

- Secondary hierarchy for read-only user data
- Usually largest directory on system
- Key subdirectories:
 - **/usr/bin/** - User command binaries
 - **/usr/sbin/** - System admin binaries
 - **/usr/lib/** - Libraries
 - **/usr/local/** - Locally installed software
 - **/usr/share/** - Architecture-independent data
 - **/usr/src/** - Source code

Directory Summary Table

Directory	Type	Purpose	Examples
/bin	Essential	User commands	ls, cp, bash
/boot	System	Boot files	vmlinuz, grub/
/dev	Virtual	Device files	sda, null, tty
/etc	System	Configurations	passwd, fstab
/home	User	User directories	user1/, user2/
/lib	System	Shared libraries	libc.so
/media	Mount	Removable media	USB/, DVD/
/opt	Application	Third-party apps	google/, custom/
/proc	Virtual	Process info	cpuinfo, meminfo
/root	User	Root home	Root's files
/tmp	Temporary	Temp files	App temp data
/usr	Application	User programs	bin/, lib/, share/
/var	Data	Variable data	log/, cache/, www/



Common Command-Line Operations

```

# Navigation
pwd                                # Current directory
ls -la                             # List all files with details
cd /etc                            # Change directory
cd ~                               # Go home

# File viewing
cat filename                       # Display file
less filename                      # Page through file
tail -f /var/log/syslog           # Follow log in real-time

# File search
find /etc -name "*.conf"
locate filename

# System info
df -h                             # Disk space
du -sh /var/log                   # Directory size
lsblk                             # Block devices
ps aux                            # All processes
  
```

References

- Free Software Foundation. (n.d.). *GNU GRUB Manual 2.12*. GNU Project. Retrieved November 14, 2025, from https://www.gnu.org/software/grub/manual/grub/grub.html?utm_source=chatgpt.com
- Hoffman, C. (n.d.). *What Is UEFI*. How-To Geek. <https://www.howtogeek.com/56958/htg-explains-how-uefi-will-replace-the-bios/>
- kernel.org. (n.d.). *The Linux/x86 Boot Protocol*. kernel.org. Retrieved November 14, 2025, from https://www.kernel.org/doc/Documentation/x86/boot.txt?utm_source=chatgpt.com
- Linux Foundation. (2015, March 19). *Filesystem Hierarchy Standard*. Linux Foundation. Retrieved November 14, 2025, from https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html
- Linux Journey. (n.d.). *The Shell - Command Line | Linux Journey*. LabEx. Retrieved November 14, 2025, from <https://labex.io/lesson/the-shell>